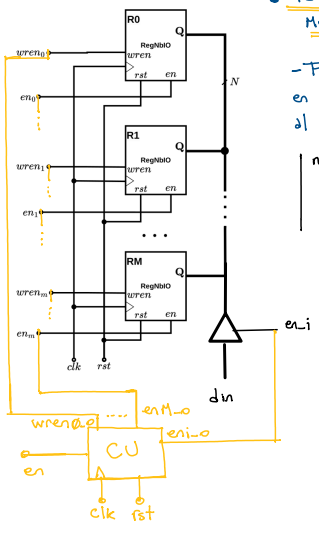
FSM

jueves, octubre 02, 2025 8:53 AM



- Recordando el ejercicio de movimiento de datos entre registros:

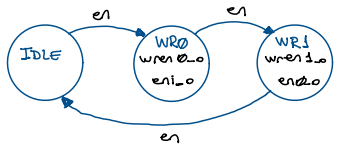
-Para guardar un dato de entrada en el registro 0 y luego moverlo al 1, teníamos:

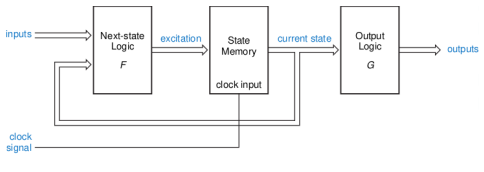
```
mov_in_to_0_1:
    en_i  wren0   ; guardamos el dato de entrada en R0
    en0   wren1   ; guardamos el dato de R0 en R1
```

. Podemos sintetizar un sistema que controle las señales de forma adecuada?

¿Si! → Una FSM

Diagrama de estados:





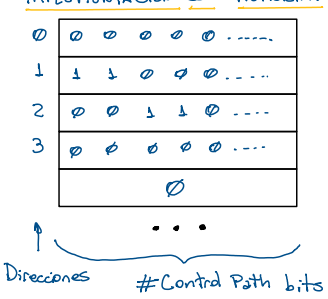
- Codifiquemos los estados:

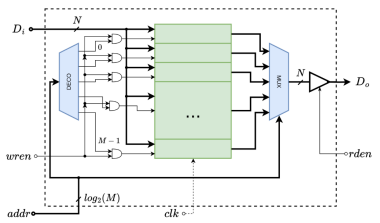
IDLE: 00
WR0: 01
WR1: 10

Lógica de salida

IN	OUTS				
current_state	eni_o	wren0_o	en0_o	wren1_o	en1_o ....
00	0	0	0	0	0
01	1	1	0	0	0 ....
10	0	0	1	1	0 ....
11	(?) => ∅				

Implementación con memoria:



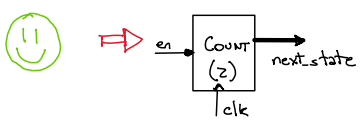


Lógica de estado siguiente:

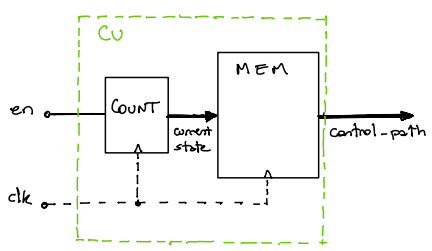
- Si 'en' está activa se avanza al estado siguiente inmediato
  . Con la codificación seleccionada:

00 → 01 → 10

¿Es un contador hasta 2?



Visión estructural de nuestra FSM 'CU':

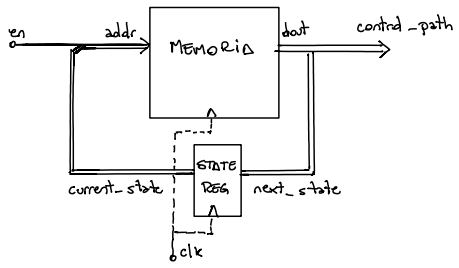


. Podemos incluir la lógica de estado siguiente también en la memoria?

Si!

FSM totalmente microprogramada:

En nuestro ejemplo quedaría:




```
assign addr = {current_state, en};
assign next_state = dout[1:0];
assign control_path = dout[C:2];
```

Y la memoria:

		control_path					
addr	eni_o	wren0_o	en0_o	wren1_o	....	<next_state>	
0	0	0	0	0	.....	0	0
1	0	0	0	0	....	0	1
2	1	1	0	0		0	1
3	1	1	0	0		1	0
4	0	0	1	1		1	0
5	0	0	1	1		0	0

Creció en ancho y profundidad

Además de todo ahora podemos generar herramientas: